

МИНИСТЕРСТВО НАУКИ И ВЫСШЕГО ОБРАЗОВАНИЯ РОССИЙСКОЙ ФЕДЕРАЦИИ  
федеральное государственное автономное образовательное учреждение высшего образования  
«САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ  
АЭРОКОСМИЧЕСКОГО ПРИБОРОСТРОЕНИЯ»

КАФЕДРА КОМПЬЮТЕРНЫХ ТЕХНОЛОГИЙ И ПРОГРАММНОЙ ИНЖЕНЕРИИ  
(КАФЕДРА 43)

КУРСОВАЯ РАБОТА (ПРОЕКТ)  
ЗАЩИЩЕНА С ОЦЕНКОЙ  
РУКОВОДИТЕЛЬ

\_\_\_\_\_  
доцент, к.т.н  
должность, уч. степень, звание

\_\_\_\_\_  
подпись, дата

\_\_\_\_\_  
А.А. Попов  
инициалы, фамилия

ПОЯСНИТЕЛЬНАЯ ЗАПИСКА  
К КУРСОВОЙ РАБОТЕ (ПРОЕКТУ)

**Программирование встроенного приложения**

по дисциплине: Программирование встроенных приложений

РАБОТУ ВЫПОЛНИЛ

СТУДЕНТ гр. № \_\_\_\_\_ 4131з

\_\_\_\_\_  
подпись, дата

\_\_\_\_\_  
М.Д. Быстров  
инициалы, фамилия

Санкт-Петербург 2026

## Оглавление

|                                              |    |
|----------------------------------------------|----|
| 1. Задание на курсовое проектирование .....  | 3  |
| 2. Техническое задание на прибор .....       | 4  |
| 2.1 Основные требования .....                | 5  |
| 3. Проектирование архитектуры системы .....  | 7  |
| 3.1 Структурная схема .....                  | 7  |
| 3.2 Диаграмма состояний .....                | 10 |
| 3.4 Диаграмма последовательности .....       | 11 |
| 3.5 Диаграмма деятельности .....             | 13 |
| 3.6 Диаграмма классов (сервер) .....         | 15 |
| 4 Разработка программного обеспечения .....  | 16 |
| 4.1 Инициализация LCD-дисплея .....          | 16 |
| 4.2 Сканирование клавиатуры .....            | 17 |
| 4.3 Работа с UART .....                      | 18 |
| 4.4 Основной цикл программы .....            | 19 |
| 4.5 Парсер выражений (сервер) .....          | 20 |
| 4.6 Обработка запросов на сервере .....      | 23 |
| 5. Контрольные испытания .....               | 26 |
| 6. Экономическая оценка .....                | 30 |
| 7. Заключение .....                          | 31 |
| Список использованной литературы .....       | 32 |
| Приложение 1. Исходный код .....             | 33 |
| A.1 Код микроконтроллера (main.c) .....      | 33 |
| A.2 Код сервера (CalculatorServer.pde) ..... | 42 |

## 1. Задание на курсовое проектирование

Вариант 5. Удалённые вычисления на калькуляторе выражений.

Реализовать в Proteus удалённого клиента инженерного калькулятора выражений, который производит вычисления на сервере в Processing. Ввод выражения для вычисления осуществляется, используя кнопки и LCD, далее выражение по интерфейсу UART передаётся на сервер-ПК, где вычисляется и сохраняется в журнале вычислений, а результат передаётся обратно в калькулятор и отображается на экране. Вводимые символы: '0', '1', '2', '3', '4', '5', '6', '7', '8', '9', '.', '=', '+', '-', '\*', '/', '(', ')', 'ln'. Журнал вычислений должен быть доступен для просмотра в Processing, и при выборе выражения передавать результат его вычисления клиенту для просмотра. Не указанные ограничения и форматы доопределить самостоятельно.

## 2. Техническое задание на прибор

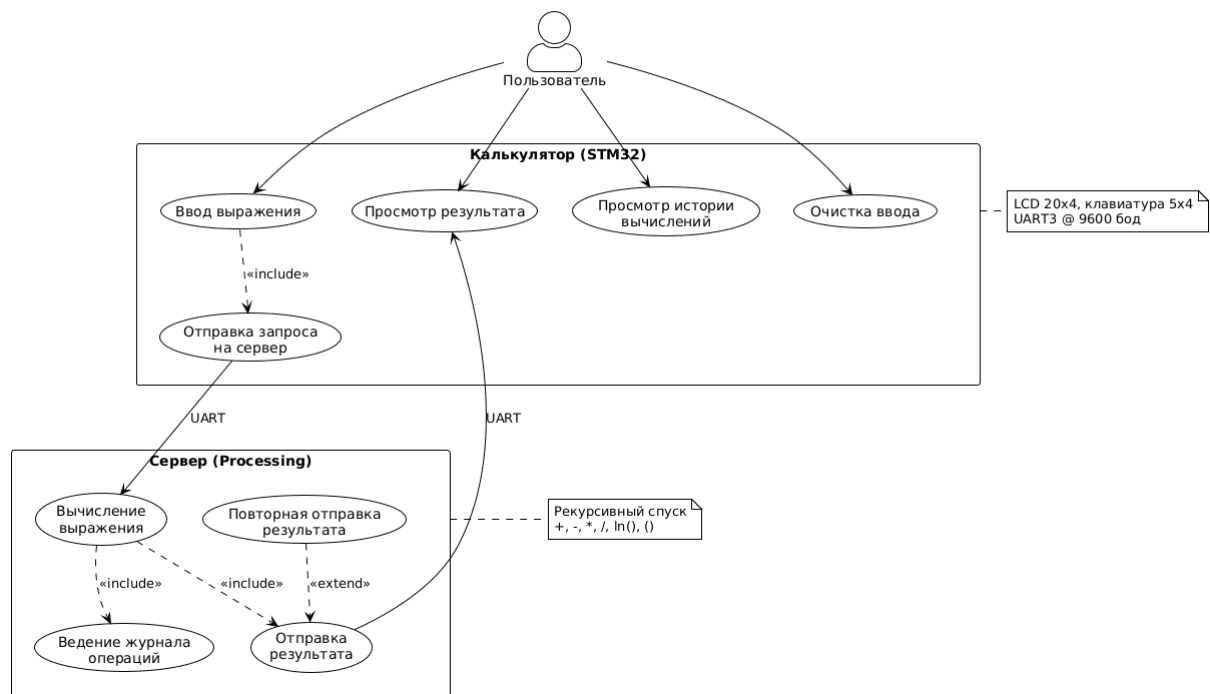
Необходимо разработать калькулятор выражений с вычислением на удаленном сервере математических расчетов.

В качестве клиентской части выступает микроконтроллер STM32F103C8 с подключенным периферийным оборудованием, в качестве серверной части – среда Processing. Связь между сервером и клиентом должна осуществляться посредством взаимодействия по интерфейсу UART. Клиентская часть (микроконтроллер и подключенная периферия) должна эмулироваться в среде Proteus.

Система удалённых вычислений предназначена для выполнения математических расчётов с использованием клиент-серверной архитектуры. Клиентская часть должна быть реализована на языке C, серверная — на языке Java.

Пользователь может вводить выражения, просматривать результаты на сервере, отправлять результат из журнала на дисплей, очищать ввод. Микроконтроллер обрабатывает нажатия клавиш и взаимодействует с сервером по UART. Сервер вычисляет выражения и ведёт журнал операций.

# **Диаграмма вариантов использования** **Удалённые вычисления на калькуляторе выражений**



*Рисунок 1 Диаграмма вариантов использования*

## **2.1 Основные требования**

Таблица 1 – Технические требования к системе

| Параметр                | Требование                                   |
|-------------------------|----------------------------------------------|
| Микроконтроллер         | STM32F103C8T6 (ARM Cortex-M3, 72 МГц)        |
| Дисплей                 | LCD 20x4 символов (LM044L), 4-битный режим   |
| Устройство ввода        | Матричная клавиатура 5x4 (20 клавиш)         |
| Интерфейс связи         | UART3 (PB10-TX, PB11-RX)                     |
| Скорость передачи       | 9600 бод, 8N1                                |
| Протокол                | Текстовый: <выражение>\r\n → <результат>\r\n |
| Поддерживаемые операции | +, -, *, /, ln(), ()                         |
| Сервер                  | Processing 4.x                               |
| Симулятор               | Proteus 8.x                                  |

Система должна обеспечивать корректное вычисление математических выражений с учётом приоритета операций и скобок.

Таблица 2 – Назначение клавиш клавиатуры

|   |   |               |                |
|---|---|---------------|----------------|
| 1 | 2 | 3             | + - сложение   |
| 4 | 5 | 6             | - - вычитание  |
| 7 | 8 | 9             | * - умножение  |
| . | 0 | ln – логарифм | C – очистка    |
| ( | ) | / - деление   | = - вычисление |

### 3. Проектирование архитектуры системы

#### 3.1 Структурная схема

Структурная схема системы показывает взаимодействие аппаратных и программных компонентов. Подключено следующее оборудование: LCD-дисплей 20x4 символов (контроллер HD44780), матричная клавиатура 5x4 (20 клавиш) и виртуальный COM-порт (COMPIM) для связи с сервером.



Рисунок 4 Структурная схема подключения периферийных устройств к МК

Для настройки работы портов GPIOA, GPIOB используется регистр RCC\_APB2ENR (APB2 peripheral clock enable register) — он включает тактирование периферии.

Для настройки выводов LCD (PA0-PA5) используется регистр GPIOA\_CRL (Port configuration register low). Каждый пин настраивается 4 битами: MODE[1:0] определяет режим (вход/выход и скорость), CNF[1:0] — конфигурацию (push-pull, open-drain и т.д.).

Для настройки клавиатуры используются регистры GPIOB\_CRL и GPIOB\_CRH:

- PB0-PB4 (строки) — Output Push-Pull для последовательной активации строк;
- PB5-PB8 (столбцы) — Input Pull-Up для чтения состояния нажатых клавиш.

Для настройки UART3 используются регистры:

- GPIOB\_CRH — настройка PB10 как Alternate Function Push-Pull (TX), PB11 как Input Floating (RX);
- USART3\_BRR (Baud rate register) — делитель частоты для получения скорости 9600 бод;
- USART3\_CR1 (Control register 1) — включение UART, передатчика и приёмника.

Расчёт значения регистра BRR для скорости 9600 бод при тактовой частоте 8 МГц:

$$BRR = f_{CLK} / (16 \times \text{baud}) = 8000000 / (16 \times 9600) = 52.08 \approx 52 = 0x34$$

С учётом дробной части: USARTDIV = 52.08, мантисса = 52 = 0x34, дробная часть =  $0.08 \times 16 = 1.28 \approx 1$ . Итого: BRR = 0x341.

Для работы системы будем использовать автомат состояний. Требования к реальному времени не являются критичными для калькулятора.



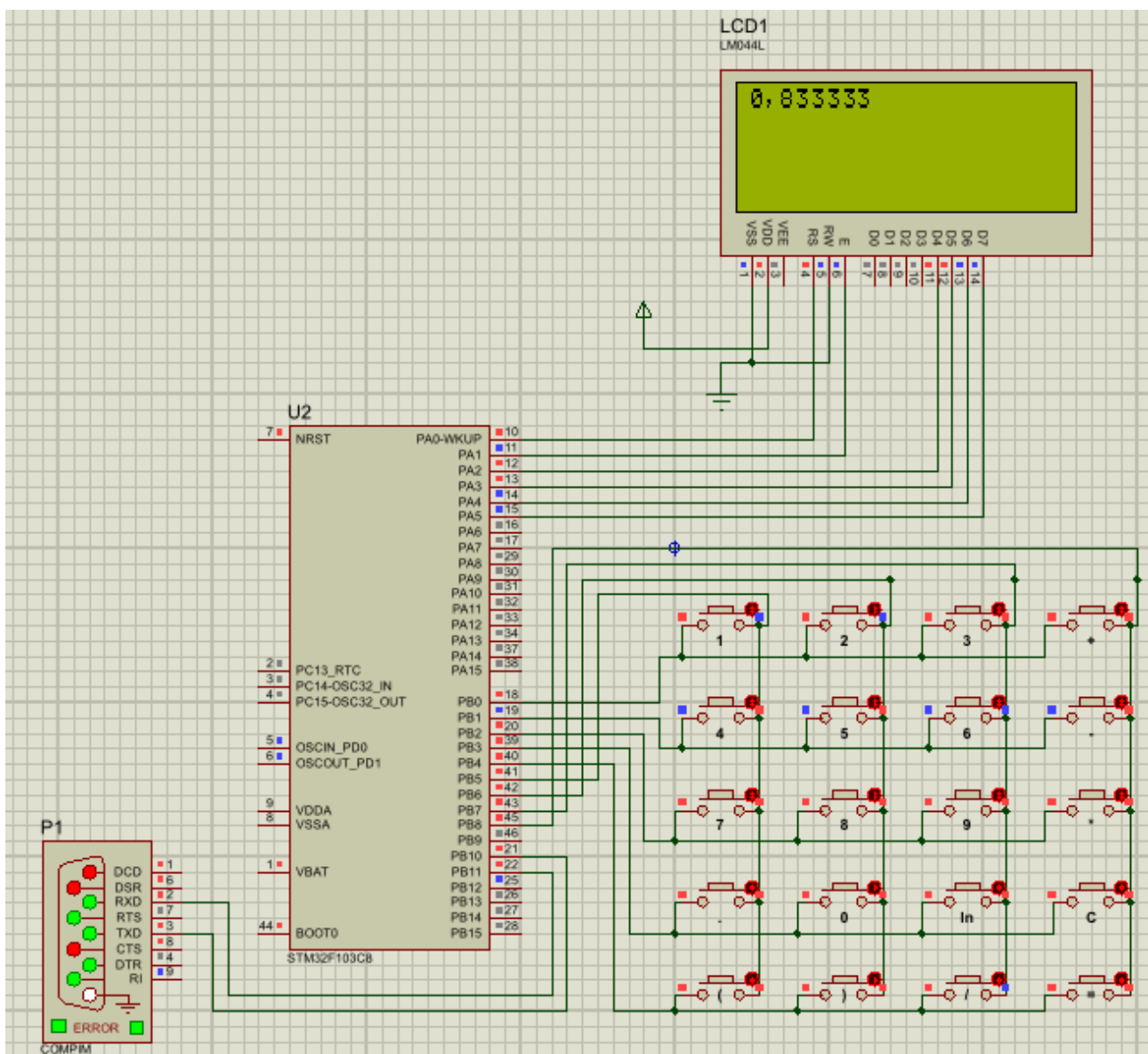


Рисунок 3 Скриншот спроектированной схемы в Proteus

### 3.2 Диаграмма состояний

Изначально, после сброса, система находится в неинициализированном состоянии (STATE\_START). Далее происходит инициализация периферии (LCD, клавиатура, UART), и система переходит в состояние ожидания ввода (STATE\_INPUT).

#### Диаграмма состояний МК STM32 Удалённые вычисления на калькуляторе выражений

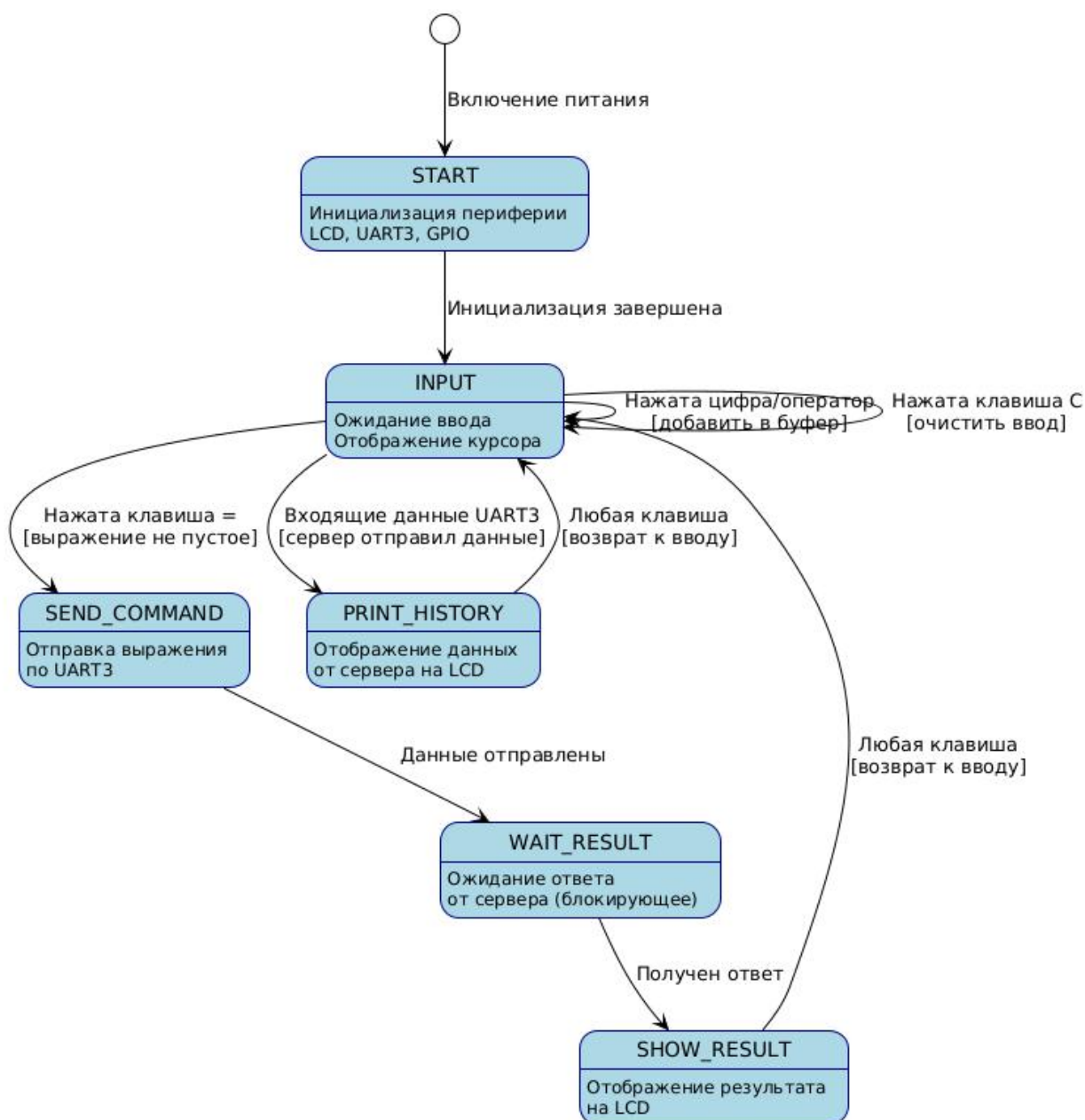


Рисунок 5. – Диаграмма состояний микроконтроллера

При поступлении внешнего сигнала (ввод с клавиатуры, клавиша "=") система переходит в состояние отправки команды (STATE\_SEND\_COMMAND), затем в состояние ожидания результата (STATE\_WAIT\_RESULT). После получения ответа от сервера система переходит в состояние отображения результата (STATE\_SHOW\_RESULT). При ошибке — в состояние STATE\_ERROR.

Для состояний можно сделать следующие выводы:

- 1) Система имеет 7 состояний: START, INPUT, SEND\_COMMAND, WAIT\_RESULT, SHOW\_RESULT, ERROR, PRINT\_HISTORY.
- 2) Переходы между состояниями определяются событиями: нажатие клавиш, получение данных по UART, таймаут.
- 3) Из любого состояния отображения (SHOW\_RESULT, ERROR, PRINT\_HISTORY) можно вернуться в состояние INPUT нажатием любой клавиши.

### **3.4 Диаграмма последовательности**

Рассмотрим работу системы на обобщённой диаграмме последовательности взаимодействия. Диаграмма показывает типичный сценарий: пользователь вводит выражение, нажимает "=", микроконтроллер отправляет выражение на сервер, сервер вычисляет результат и отправляет его обратно.

**Диаграмма последовательности**  
**Процесс вычисления выражения**

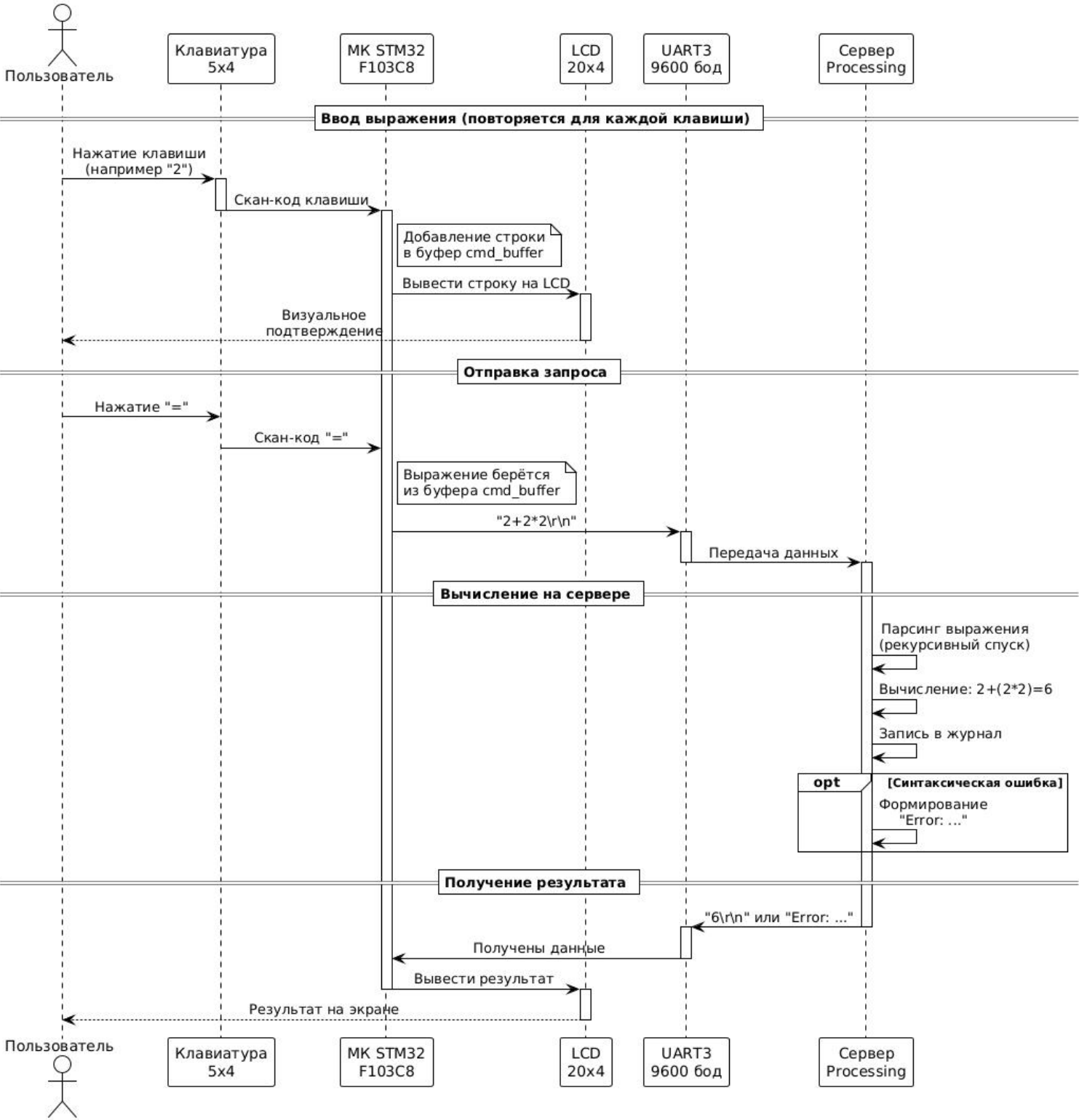


Рисунок 6. – Диаграмма последовательности вычисления выражения

Из диаграммы последовательности можно заключить:

- 1) Клавиатура работает в режиме опроса — микроконтроллер сканирует состояние клавиш.
- 2) UART работает в блокирующем режиме — после отправки выражения микроконтроллер ожидает ответ.
- 3) Протокол обмена текстовый: запрос и ответ завершаются символами `\r\n`.
- 4) Сервер ведёт журнал всех операций с возможностью повторной отправки результата.

### **3.5 Диаграмма деятельности**

Детализируем состояния системы на диаграмме деятельности функции `main`. Диаграмма показывает основной цикл программы: инициализация, бесконечный цикл опроса клавиатуры и UART.

### **Диаграмма деятельности** **Основной цикл программы МК**

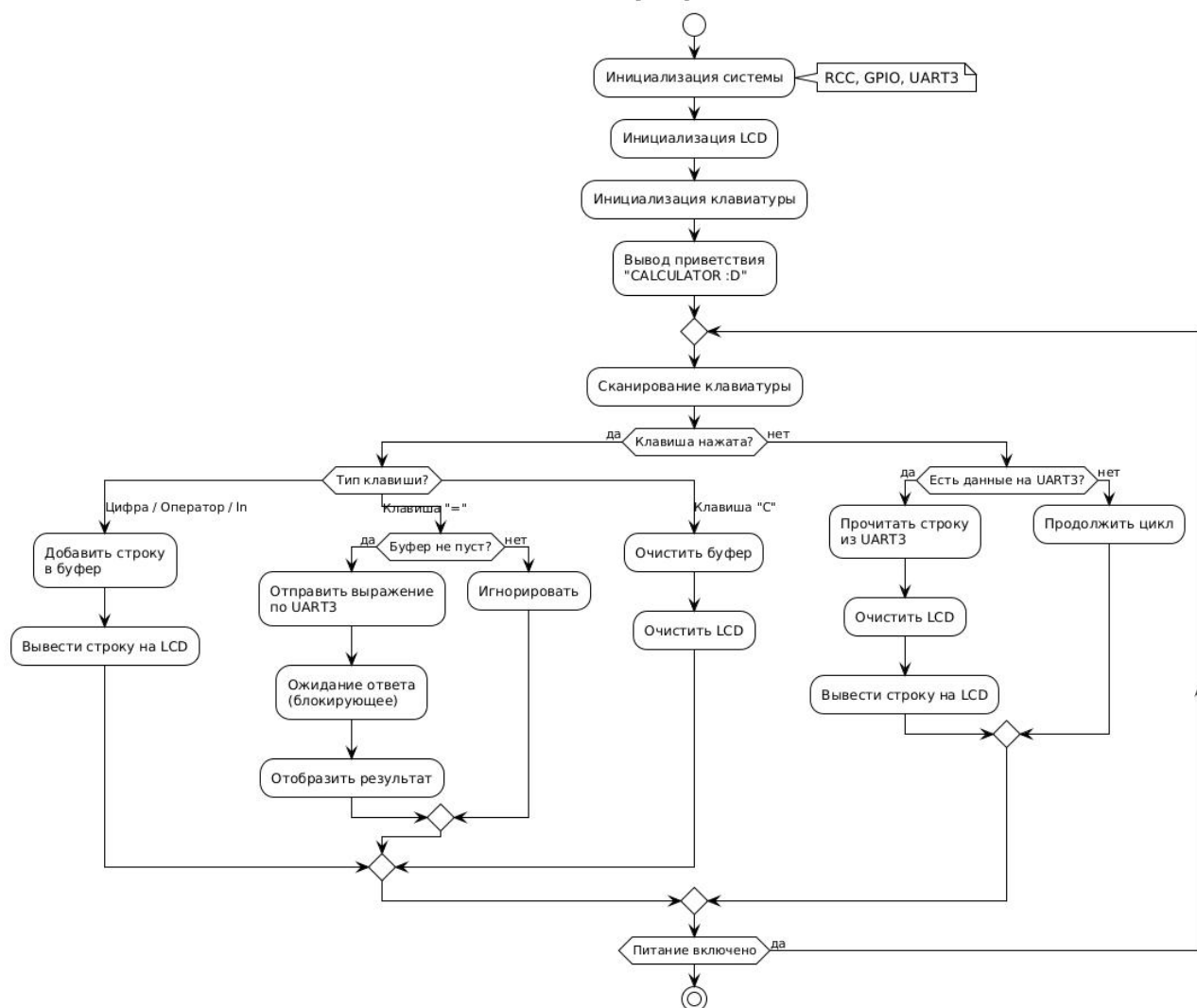


Рисунок 7. – Диаграмма деятельности функции main

Отсюда можно выделить следующие основные модули программы МК:

- 1) Основной цикл программы;
- 2) Инициализация LCD-дисплея;
- 3) Инициализация GPIO для матричной клавиатуры;
- 4) Инициализация UART;
- 5) Сканирование матричной клавиатуры;
- 6) Отправка выражения на сервер и получение результата (UART).

### 3.6 Диаграмма классов (сервер)

Диаграмма классов описывает структуру серверного приложения на Processing. Основной класс CalculatorServer управляет последовательным портом, журналом операций и графическим интерфейсом.



Рисунок 8. – Диаграмма классов серверного приложения

#### Выводы по диаграмме классов:

- 1) Класс JournalEntry — структура данных для хранения записи журнала (выражение, результат, временная метка).
- 2) Статический класс Parser реализует рекурсивный спуск для разбора математических выражений.
- 3) Грамматика парсера поддерживает операции +, -, \*, /, функцию ln() и скобки.
- 4) Приоритет операций: скобки → ln() → \* и / → + и -.

## 4 Разработка программного обеспечения

В данном разделе рассмотрены основные функции программного обеспечения микроконтроллера и сервера. Для каждой функции приводится описание алгоритма работы, используемых регистров и примеры кода.

### 4.1 Инициализация LCD-дисплея

LCD-дисплей LM044L (20x4 символов) использует контроллер HD44780. Для экономии выводов микроконтроллера применяется 4-битный режим работы, при котором данные передаются по 4 линиям (D4-D7), а не по 8.

Подключение LCD к микроконтроллеру:

- PA0 — RS (Register Select): 0 — команда, 1 — данные;
- PA1 — EN (Enable): строб записи данных;
- PA2-PA5 — D4-D7 (линии данных в 4-битном режиме).

Функция LCD\_Init выполняет инициализацию LCD-дисплея согласно спецификации HD44780. Сначала включается тактирование порта GPIOA через регистр RCC\_APB2ENR, затем настраиваются пины PA0-PA5 как выходы Push-Pull (MODE=01, CNF=00). После этого выполняется последовательность инициализации контроллера.

```
void LCD_Init(void) {  
    // Включаем тактирование порта A  
    RCC->APB2ENR |= RCC_APB2ENR_IOPAEN;  
  
    // Настраиваем PA0-PA5 как Output Push-Pull (10MHz)  
    GPIOA->CRL &= ~(0x00FFFFFF);  
    GPIOA->CRL |= (0x00111111);  
  
    // Последовательность инициализации 4-бит  
    delay_us(20000); // Ожидание включения питания (>15мс)  
    LCD_SendNibble(0x03, 0); delay_us(5000);  
    LCD_SendNibble(0x03, 0); delay_us(200);  
    LCD_SendNibble(0x03, 0);  
    LCD_SendNibble(0x02, 0); // Переход на 4-битный интерфейс  
  
    // Настройка контроллера  
    LCD_Cmd(0x28); // 4-бит, 2(4) строки, шрифт 5x8  
    LCD_Cmd(0x0C); // Дисплей вкл, курсор выкл  
    LCD_Cmd(0x06); // Авто-инкремент курсора  
    LCD_Cmd(0x01); // Очистка дисплея  
}
```



Функция LCD\_SendNibble отправляет 4 бита данных на LCD. Пин RS определяет тип данных: 0 — команда, 1 — данные для отображения.

```
void LCD_SendNibble(uint8_t nibble, uint8_t rs) {
    if(rs) GPIOA->BSRR = GPIO_BSRR_BS0; // RS = 1 (данные)
    else   GPIOA->BSRR = GPIO_BSRR_BR0; // RS = 0 (команда)

    // Выводим PA2-PA5 младшие 4 бита данных
    GPIOA->ODR = (GPIOA->ODR & ~(0xF << 2)) | ((nibble & 0x0F) << 2);
    LCD_Pulse();
}
```

## 4.2 Сканирование клавиатуры

Матричная клавиатура 5x4 подключена к порту GPIOB. Для её работы необходимо настроить пины PB0-PB4 (строки) как выходы Push-Pull, а PB5-PB8 (столбцы) как входы с подтяжкой к питанию (Pull-Up).

### Конфигурация GPIO для клавиатуры:

- PB0-PB4: MODE=01 (Output 10MHz), CNF=00 (Push-Pull);
- PB5-PB8: MODE=00 (Input), CNF=10 (Input with Pull-Up/Pull-Down).

Алгоритм сканирования основан на последовательной активации строк. Изначально все строки установлены в высокий уровень (1). При сканировании каждая строка по очереди устанавливается в низкий уровень (0). Если в этот момент нажата клавиша в данной строке, соответствующий столбец также примет низкий уровень, что будет считано микроконтроллером.

Функция Scan\_Keypad реализует сканирование матричной клавиатуры 5x4. Каждая строка последовательно устанавливается в 0, затем считываются состояния столбцов. При обнаружении нажатия возвращается индекс клавиши (0-19). Если ни одна клавиша не нажата, возвращается значение 0xFF.

```
uint8_t Scan_Keypad(void) {
    for (uint8_t r = 0; r < 5; r++) {
        // Активируем строку r (устанавливаем 0)
        GPIOB->ODR = (GPIOB->ODR | 0x1F) & ~(1 << r);

        // Читаем столбцы PB5-PB8
        uint16_t cols = (GPIOB->IDR >> 5) & 0x0F;

        if (cols != 0x0F) { // Если какая-то кнопка нажата
            for (uint8_t c = 0; c < 4; c++) {
                if (!(cols & (1 << c))) { // Находим столбец
                    return (r * 4) + c; // Возвращаем индекс
                }
            }
        }
    }
    return 0xFF;
}
```

```

        }
    }
}
return 0xFF; // Ничего не нажато
}

```

### 4.3 Работа с UART

Для связи с сервером используется интерфейс UART3 микроконтроллера STM32F103. Пины PB10 (TX) и PB11 (RX) подключены к виртуальному COM-порту через компонент COMPIМ в Proteus.

**Настройка UART3 включает следующие этапы:**

- 1) Включение тактирования: порт GPIOB (RCC\_APB2ENR), модуль AFIO (RCC\_APB2ENR), модуль USART3 (RCC\_APB1ENR).
- 2) Настройка GPIO: PB10 как Alternate Function Push-Pull (CNF=10, MODE=11), PB11 как Input Floating (CNF=01, MODE=00).
- 3) Настройка скорости: регистр USART3\_BRR определяет делитель частоты. При 8 МГц и 9600 бод: BRR = 0x341.
- 4) Включение модуля: бит UE (UART Enable), TE (Transmitter Enable), RE (Receiver Enable) в регистре USART3\_CR1.

Функция UART3\_Init настраивает UART3 для связи с сервером. Используются пины PB10 (TX) и PB11 (RX). Скорость передачи — 9600 бод, формат 8N1 (8 бит данных, без чётности, 1 стоп-бит).

```

void UART3_Init(void) {
    // Включаем тактирование
    SET_BIT(RCC->APB2ENR, RCC_APB2ENR_IOPBEN | RCC_APB2ENR_AFIOEN);
    SET_BIT(RCC->APB1ENR, RCC_APB1ENR_USART3EN);

    // PB10 (TX) - Alt. Function Push-Pull
    MODIFY_REG(GPIOB->CRH, (GPIO_CRH_MODE10 | GPIO_CRH_CNF10),
                (GPIO_CRH_MODE10_1 | GPIO_CRH_MODE10_0 | GPIO_CRH_CNF10_1));

    // PB11 (RX) - Input Floating
    MODIFY_REG(GPIOB->CRH, (GPIO_CRH_MODE11 | GPIO_CRH_CNF11),
                (GPIO_CRH_CNF11_0));

    // Baudrate 9600 при 8 МГц
    WRITE_REG(USART3->BRR, 0x341);

    // Включаем UART, передатчик и приёмник
}

```

```

    SET_BIT(USART3->CR1, USART_CR1_UE | USART_CR1_TE | USART_CR1_RE);
}

```

Функция `UART3_ReceiveLine` получает строку до символов `\r\n`:

```

void UART3_ReceiveLine(char* buffer, uint16_t buffer_size) {
    char prev_sym = 0;
    while (1) {
        char sym = UART3_ReceiveChar();

        if (sym != '\r' && sym != '\n') {
            *buffer++ = sym;
            if (--buffer_size < 1) break;
        }

        if (sym == '\n' && prev_sym == '\r') break;
        prev_sym = sym;
    }
    while (buffer_size-- > 0) *buffer++ = '\0';
}

```

Протокол обмена данными — текстовый. Запрос от клиента:

<выражение>\r\n. Ответ от сервера: <результат>\r\n. Функция

`UART3_ReceiveLine` считывает символы до получения последовательности `\r\n`.

#### 4.4 Основной цикл программы

Основной цикл программы производит опрос клавиатуры и наличия порта UART на предмет входящих данных. Переменная `app_state` хранит текущее состояние системы и определяет реакцию на события.

##### Состояния системы:

- `STATE_INPUT`: ожидание ввода с клавиатуры, символы добавляются в буфер;
- `STATE_SEND_COMMAND`: отправка выражения на сервер по UART;
- `STATE_WAIT_RESULT`: ожидание ответа от сервера;
- `STATE_SHOW_RESULT`: отображение результата на LCD;
- `STATE_ERROR`: отображение ошибки;
- `STATE_PRINT_HISTORY`: вывод истории операций.

Основной цикл программы выполняет сканирование клавиатуры, обработку нажатий и взаимодействие с UART. При нажатии клавиши "=" выражение отправляется на сервер.

```

while(1) {
    uint8_t key = Scan_Keypad();

```

```

if (key != 0xFF) {
    while (key == Scan_Keypad());

    switch (key) {
        case KEYPAD_CLEAR:
            LCD_Clear();
            Clean_Cmd_Buffer();
            break;

        case KEYPAD_EQ:
            if (cmd_buffer_pos > 0) {
                app_state = STATE_SEND_COMMAND;
                LCD_Clear();
                CalculateExpr(cmd_buffer, cmd_buffer_pos,
                             res_buffer, RES_BUFFER_LEN);
                Clean_Cmd_Buffer();
                app_state = STATE_SHOW_RESULT;
                LCD_Print(res_buffer);
            }
            break;

        default:
            if (app_state != STATE_INPUT) {
                LCD_Clear();
                app_state = STATE_INPUT;
            }
            LCD_Print(Get_Key_Char(key));
            Copy_To_Cmd_Buffer(Get_Key_Char(key));
    }
}

// Проверка входящих данных от сервера
if (UART3_HasIncomeData()) {
    app_state = STATE_PRINT_HISTORY;
    UART3_ReceiveLine(res_buffer, RES_BUFFER_LEN);
    LCD_Clear();
    LCD_Print(res_buffer);
}
}

```

### **Обработка клавиш:**

- Цифры и операторы: добавляются в буфер, отображаются на LCD;
- Клавиша "=": отправка буфера на сервер, получение и отображение результата;
- Клавиша "C": очистка буфера и дисплея;
- Клавиша "\n": добавление строки "\n" в буфер;

## **4.5 Парсер выражений (сервер)**

Сервер реализован в среде Processing на языке Java. Функция сервера — разбор и вычисление математических выражений, полученных от клиента.

Парсер выражений реализован методом рекурсивного спуска (recursive descent parser).

### Грамматика парсера в форме Бэкуса-Наура:

$\text{expr} \rightarrow \text{term} (('+' | '-') \text{term})^*$

$\text{term} \rightarrow \text{factor} (('*' | '/') \text{factor})^*$

$\text{factor} \rightarrow \text{ln}(' \text{expr} ') | '(' \text{expr} ') | \text{number}$

$\text{number} \rightarrow [0-9]^+ ('.' [0-9]^+)?$

Приоритет операций обеспечивается структурой грамматики: операции с низким приоритетом (+, -) обрабатываются на верхнем уровне (parseExpr), а операции с высоким приоритетом (\*, /) — на более глубоком уровне (parseTerm). Скобки и функция ln() имеют наивысший приоритет и обрабатываются в parseFactor.

```
double parseExpression(String input) throws Exception {
    expr = input.replaceAll("\\s+", ""); // удаляем пробелы
    pos = 0;
    double result = parseExpr();

    if (pos < expr.length()) {
        throw new Exception("syntax");
    }
    return result;
}

double parseExpr() throws Exception {
    double left = parseTerm();

    while (pos < expr.length()) {
        char op = expr.charAt(pos);
        if (op == '+') {
            pos++;
            left = left + parseTerm();
        } else if (op == '-') {
            pos++;
            left = left - parseTerm();
        } else {
            break;
        }
    }
    return left;
}

double parseTerm() throws Exception {
    double left = parseFactor();

    while (pos < expr.length()) {
        char op = expr.charAt(pos);
```

```

        if (op == '*') {
            pos++;
            left = left * parseFactor();
        } else if (op == '/') {
            pos++;
            double right = parseFactor();
            if (right == 0) throw new Exception("div/0");
            left = left / right;
        } else {
            break;
        }
    }
    return left;
}

```

**Функция parseFactor обрабатывает числа, скобки и функцию ln():**

```

double parseFactor() throws Exception {
    // Проверка ln(
    if (pos + 2 < expr.length() &&
        expr.charAt(pos) == 'l' &&
        expr.charAt(pos + 1) == 'n' &&
        expr.charAt(pos + 2) == '(') {
        pos += 3;
        double arg = parseExpr();
        if (pos >= expr.length() || expr.charAt(pos) != ')') {
            throw new Exception("syntax");
        }
        pos++;
        if (arg <= 0) throw new Exception("ln domain");
        return Math.log(arg);
    }

    // Проверка скобок
    if (pos < expr.length() && expr.charAt(pos) == '(') {
        pos++;
        double result = parseExpr();
        if (pos >= expr.length() || expr.charAt(pos) != ')') {
            throw new Exception("syntax");
        }
        pos++;
        return result;
    }

    // Унарный минус
    boolean negative = false;
    if (pos < expr.length() && expr.charAt(pos) == '-') {
        negative = true;
        pos++;
    }

    // Парсинг числа
    int start = pos;
    while (pos < expr.length() &&
        (Character.isDigit(expr.charAt(pos)) ||
        expr.charAt(pos) == '.')) {
        pos++;
    }

    if (start == pos) throw new Exception("syntax");
}

```

```

        double value = Double.parseDouble(expr.substring(start, pos));
        return negative ? -value : value;
    }

```

### **Обработка ошибок в парсере:**

- Синтаксическая ошибка (некорректное выражение): `Exception("syntax");`
- Деление на ноль: `Exception("div/0");`
- Логарифм неположительного числа: `Exception("ln domain").`

## **4.6 Обработка запросов на сервере**

Серверное приложение использует событийную модель обработки данных. Функция `serialEvent()` автоматически вызывается средой Processing при поступлении данных в буфер последовательного порта.

### **Алгоритм обработки запроса:**

- 1) Чтение строки до символа `\n` (функция `readStringUntil()`);
- 2) Удаление пробельных символов (функция `trim()`);
- 3) Разбор и вычисление выражения (функция `parseExpression()`);
- 4) Форматирование результата (целое или дробное число);
- 5) Добавление записи в журнал (класс `JournalEntry`);
- 6) Отправка результата клиенту (функция `sendResult()`).

Функция `serialEvent` вызывается при получении данных по последовательному порту. Она парсит выражение, вычисляет результат и отправляет его клиенту. В случае ошибки отправляется сообщение вида "Error: <тип ошибки>".

```

void serialEvent(Serial p) {
    String raw = p.readStringUntil('\n');
    if (raw == null) return;

    String input = raw.trim();
    if (input.length() == 0) return;

    lastReceived = input;
    println("RX: " + input);

    // Парсинг и вычисление
    String result;
    try {
        double value = parseExpression(input);
    }
}

```

```

        // Форматирование результата
        if (value == (long) value) {
            result = String.valueOf((long) value);
        } else {
            result = String.format("%.6f", value)
                .replaceAll("0+$", "")
                .replaceAll("\\.\\$", "");
        }
    } catch (Exception e) {
        result = "Error: " + e.getMessage();
    }

    // Добавляем в журнал
    journal.add(new JournalEntry(input, result));

    // Отправляем результат
    sendResult(result);
}

```

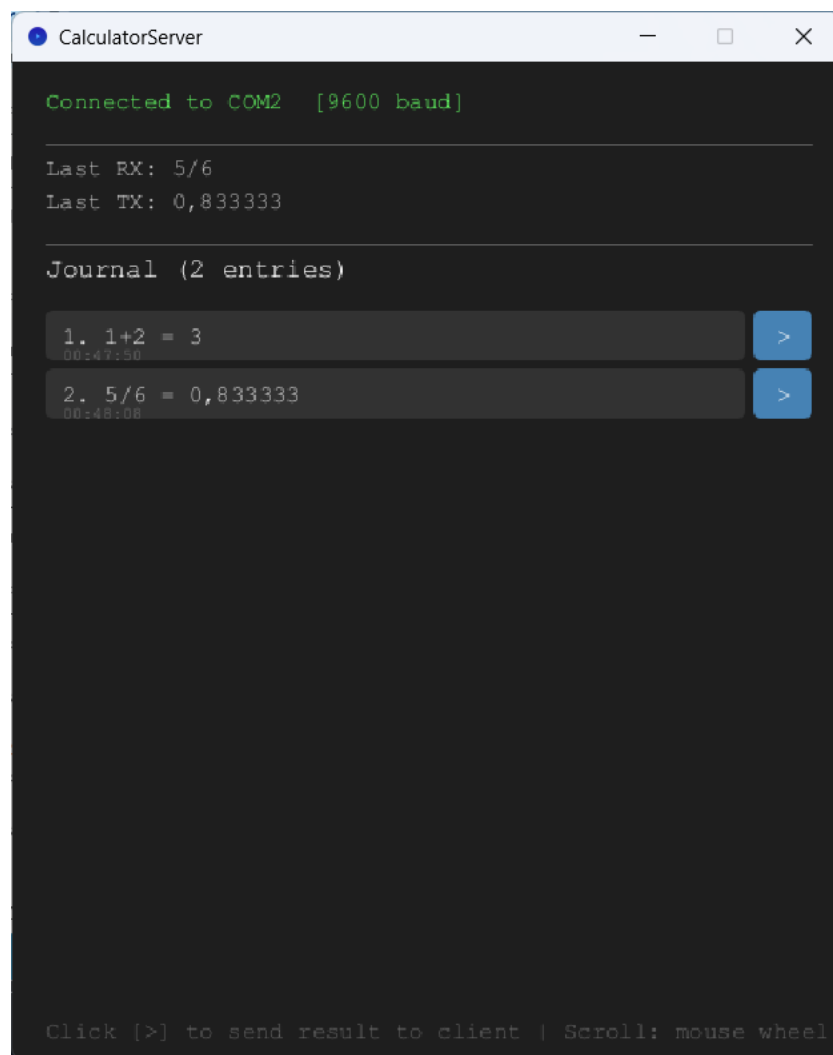
**Класс JournalEntry хранит записи журнала операций. Каждая запись содержит:**

- timestamp: время выполнения операции (формат HH:mm:ss);
- expression: исходное выражение от клиента;
- result: результат вычисления или сообщение об ошибке.

**Графический интерфейс сервера отображает:**

- Текущее состояние соединения (порт, скорость);
- Последнее полученное выражение и результат;
- Журнал всех операций с прокруткой;
- Кнопка "Resend" для повторной отправки последнего результата.





*Рисунок 2 Скриншот интерфейса сервера Processing*

## 5. Контрольные испытания

В данном разделе представлены результаты контрольных испытаний системы. Тестирование проводилось в симуляторе Proteus 8.x с использованием виртуального COM-порта (VSPE — Virtual Serial Port Emulator).

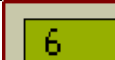
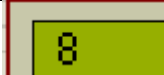
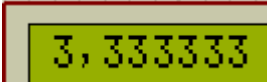
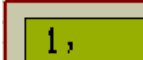
### Методика тестирования:

- 1) Запуск симуляции в Proteus с загруженной прошивкой (.hex);
- 2) Запуск серверного приложения Processing с подключением к COM2;
- 3) Создание пары виртуальных портов COM1↔COM2 в VSPE;
- 4) Ввод тестовых выражений с виртуальной клавиатуры в Proteus;
- 5) Проверка результатов на LCD-дисплее и в журнале сервера.

### Тестирование охватывает следующие категории:

- Базовые арифметические операции (+, -, \*, /);
- Приоритет операций ( $2+2*2$  должно быть 6, а не 8);
- Работа скобок  $((2+2)*2)$  должно быть 8);
- Функция натурального логарифма;
- Обработка ошибок (деление на ноль, некорректный ввод).

Таблица 3 – Результаты контрольных испытаний

| № | Действие              | Ожидаемый результат | Факт. результат                                                                       |
|---|-----------------------|---------------------|---------------------------------------------------------------------------------------|
| 1 | Ввод: 2+2             | 4                   |  |
| 2 | Ввод: 2+2*2           | 6<br>(приоритет)    |  |
| 3 | Ввод: (2+2)*2         | 8 (скобки)          |  |
| 4 | Ввод: 10/3            | 3.333333            |  |
| 5 | Ввод: ln(2.718281828) | ≈1                  |  |

|    |                                          |                  |                  |
|----|------------------------------------------|------------------|------------------|
| 6  | Ввод: 1/0                                | Error: div/0     | Error: div/0     |
| 7  | Ввод: ln(-1)                             | Error: ln domain | Error: ln domain |
| 8  | Ввод: ln                                 | Error: syntax    | Error: syntax    |
| 9  | Нажатие C                                | Очистка дисплея  |                  |
| 10 | Отправка с сервера<br>4. 10/3 = 3,333333 | Вывод на LCD     | 3,333333         |

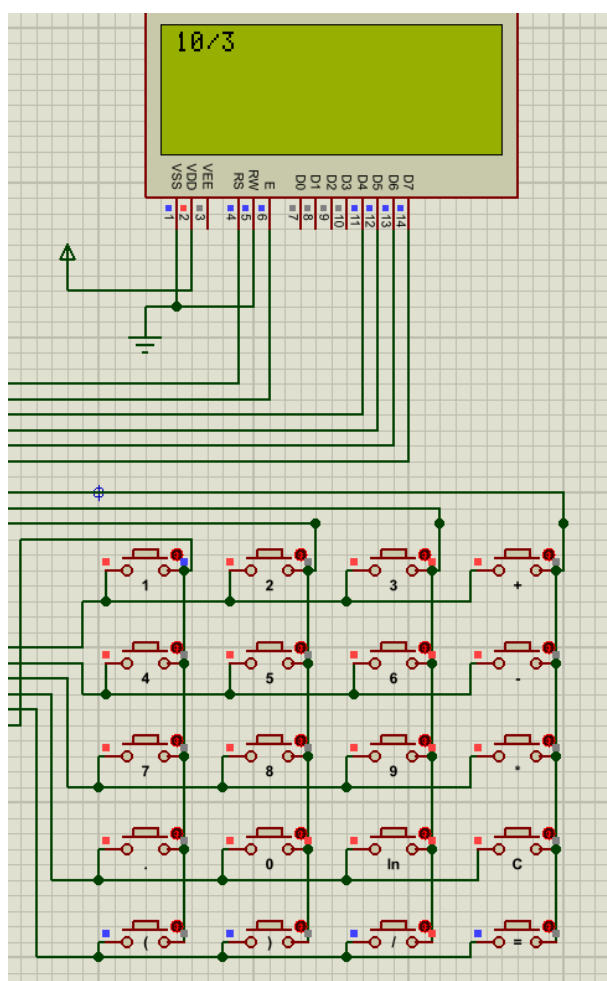


Рисунок 9. – Скриншот тестирования в Proteus (ввод выражения)

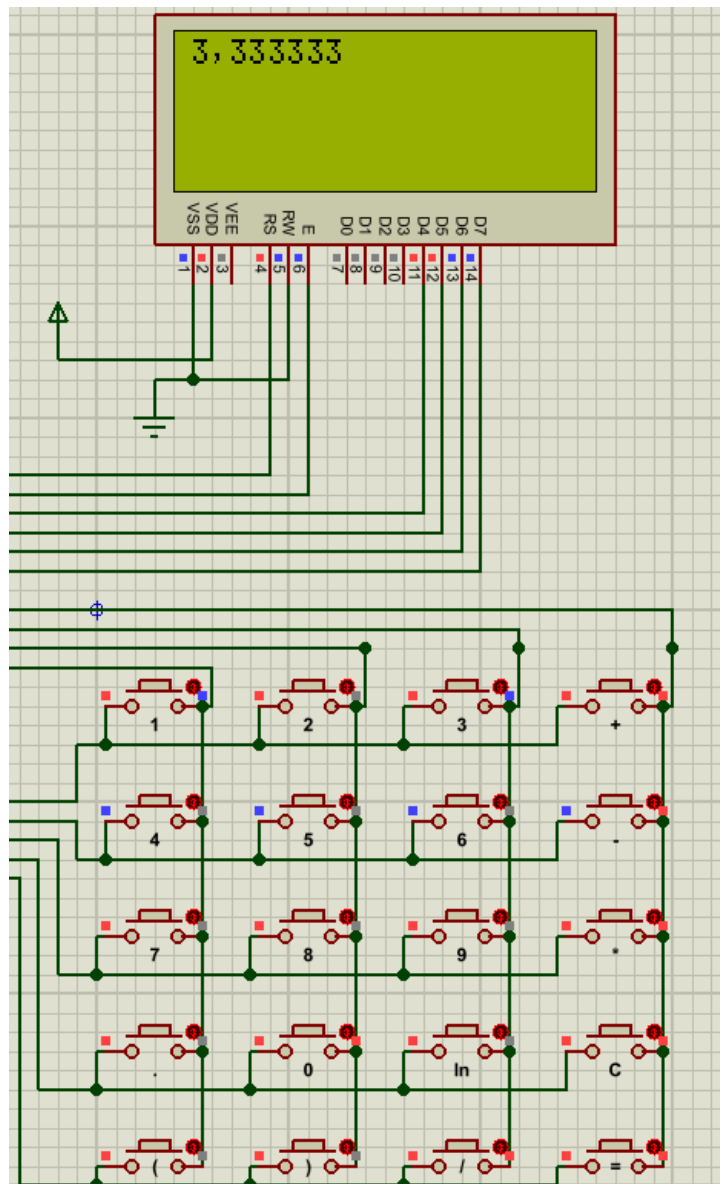


Рисунок 10. – Скриншот тестирования в Proteus (результат)

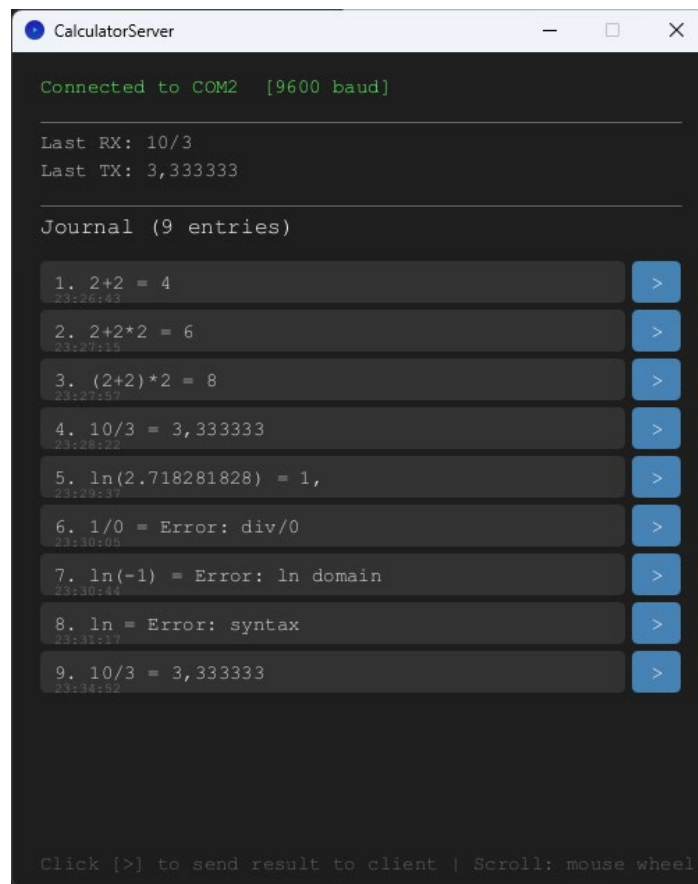


Рисунок 11. – Скриншот журнала на сервере Processing

Протестированы весь необходимый функционал калькулятора и обработка ошибок ввода.

## 6. Экономическая оценка

В данном разделе приведена оценка стоимости компонентов для реализации физического прототипа системы. Цены указаны по данным AliExpress на 2025 год.

Таблица 4 – Стоимость компонентов системы

| Компонент                     | Кол-во  | Цена, руб. | Сумма, руб. |
|-------------------------------|---------|------------|-------------|
| STM32F103C8T6<br>(Blue Pill)  | 1       | 250        | 250         |
| LCD 2004A (20x4,<br>HD44780)  | 1       | 350        | 350         |
| Матричная<br>клавиатура 4x5   | 1       | 150        | 150         |
| USB-UART<br>конвертер (CH340) | 1       | 80         | 80          |
| Макетная плата                | 1       | 100        | 100         |
| Провода, резисторы            | 1 набор | 100        | 100         |
| ST-Link V2<br>(программатор)  | 1       | 200        | 200         |
|                               |         | ИТОГО:     | 1230        |

Общая стоимость компонентов составляет около 1230 рублей.

## 7. Заключение

В ходе выполнения курсовой работы была разработана система удалённых вычислений на базе микроконтроллера STM32F103C8. Система состоит из клиентской части (калькулятор с LCD-дисплеем и клавиатурой) и серверной части (приложение на Processing).

Были решены следующие задачи:

1. Разработана схема подключения LCD-дисплея 20x4 в 4-битном режиме.
2. Реализовано сканирование матричной клавиатуры 5x4.
3. Настроен UART3 для связи с сервером на скорости 9600 бод.
4. Разработан парсер математических выражений методом рекурсивного спуска.
5. Реализован журнал операций с возможностью повторной отправки результатов.

В результате работы были получены практические навыки:

- программирования микроконтроллеров семейства STM32;
- работы с периферийными устройствами (LCD, клавиатура, UART);
- разработки клиент-серверных приложений;
- использования симулятора Proteus для отладки встраиваемых систем.

Система успешно протестирована в симуляторе Proteus и соответствует техническому заданию.

## Список использованной литературы

1. Микроконтроллеры. Разработка встраиваемых приложений: учебное пособие / А.Е. Васильев; С.-Петербургский государственный политехнический ун-т. - СПб. : Изд-во СПбГПУ, 2003. - 211 с.
2. The Definitive Guide to ARM Cortex-M3 and Cortex-M4 Processors. Third Edition. Joseph Yiu. ARM Ltd., Cambridge, UK. [электронный ресурс] // URL: <https://www.pdfdrive.com/the-definitive-guide-to-arm-cortex-m3-and-cortex-m4-processors-e187111520.html> (дата обращения 12.05.2020).
3. Джозеф Ю. Ядро Cortex-M3 компании ARM. Полное руководство. 2012. ISBN: 978- 5-94120-243-0. [электронный ресурс] // URL: <https://book.xyz/book/2373589/b5c3ad> (дата обращения 12.05.2020).
4. RM0008. Reference manual STM32F101xx, STM32F102xx, STM32F103xx, STM32F105xx and STM32F107xx advanced Arm®-based 32-bit MCUs [электронный ресурс] // URL: [https://www.st.com/resource/en/reference\\_manual/cd00171190-stm32f101xx-stm32f102xx-stm32f103xx-stm32f105xx-and-stm32f107xx-advanced-armbased-32bit-mcus-stmicroelectronics.pdf](https://www.st.com/resource/en/reference_manual/cd00171190-stm32f101xx-stm32f102xx-stm32f103xx-stm32f105xx-and-stm32f107xx-advanced-armbased-32bit-mcus-stmicroelectronics.pdf) (дата обращения 12.05.2020).
5. Datasheet STM32F103x8 STM32F103xB Medium-density performance line ARM®-based 32-bit MCU with 64 or 128 KB Flash, USB, CAN, 7 timers, 2 ADCs, 9 com. Interfaces. [электронный ресурс] // URL: <https://www.st.com/resource/en/datasheet/stm32f103c8.pdf> (дата обращения 12.05.2020).
6. Мартин М. Инсайдерское руководство по STM32 [электронный ресурс] // URL: <https://istarik.ru/file/STM32.pdf> (дата обращения 12.05.2020).
7. Рекомендация МСЭ-R М.1677-1 Международный код Морзе. Международный союз электросвязи 2009 [электронный ресурс] //URL: [https://www.itu.int/dms\\_pubrec/itu-r/rec/m/R-REC-M.1677-1-200910-I!!PDF-R.pdf](https://www.itu.int/dms_pubrec/itu-r/rec/m/R-REC-M.1677-1-200910-I!!PDF-R.pdf) (дата обращения 12.05.2020)



## Приложение 1. Исходный код

### А.1 Код микроконтроллера (main.c)

```
#include "stm32f10x.h"

// keypad button indexes
#define KEYPAD_1 0
#define KEYPAD_2 1
#define KEYPAD_3 2
#define KEYPAD_4 4
#define KEYPAD_5 5
#define KEYPAD_6 6
#define KEYPAD_7 8
#define KEYPAD_8 9
#define KEYPAD_9 10
#define KEYPAD_0 13
#define KEYPAD_PLUS 3
#define KEYPAD_MINUS 7
#define KEYPAD_DIV 18
#define KEYPAD_LN 14
#define KEYPAD_MUL 11
#define KEYPAD_EQ 19
#define KEYPAD_CLEAR 15
#define KEYPAD_OPEN_BR 16
#define KEYPAD_CLOSE_BR 17
#define KEYPAD_DOT 12

// app states
#define STATE_START 0 //user hello
#define STATE_INPUT 1 //wait user input
#define STATE_SEND_COMMAND 2 // send command to server
#define STATE_WAIT_RESULT 3 // wait result from server
#define STATE_SHOW_RESULT 4 // result printed to screen
#define STATE_ERROR 5 // error state
#define STATE_PRINT_HISTORY 6 // print history from server

char* keypad_chars[20] = {
    "1", "2", "3", "+",
    "4", "5", "6", "-",
    "7", "8", "9", "*",
    ".", "0", "\n", "C",
    "(", ")", "/", "="};

// string buffers for UART send-receive
#define CMD_BUFFER_LEN 80
#define RES_BUFFER_LEN 80
```

```

char cmd_buffer[CMD_BUFFER_LEN];
char cmd_buffer_pos = 0;
char res_buffer[RES_BUFFER_LEN];

// row addresses in lcd screen
const uint8_t row_addresses[] = {0x00, 0x40, 0x14, 0x54};
uint8_t cur_row = 0, cur_col = 0;

uint8_t app_state = STATE_START;

void delay_us(uint32_t us) {
    // delay in ns for frequency 8MHz
    uint32_t count = us * 2;
    while(count--) {
        __NOP();
    }
}

// toggle EN LCD pin for command executing
void LCD_Pulse(void) {
    GPIOA->BSRR = GPIO_BSRR_BS1; // EN = 1
    delay_us(5);                 // Wait for LCD
    GPIOA->BSRR = GPIO_BSRR_BR1; // EN = 0
    delay_us(40);                // Wait for LCD
}

// send 4-bit data to HD44780
void LCD_SendNibble(uint8_t nibble, uint8_t rs) {
    if(rs) GPIOA->BSRR = GPIO_BSRR_BS0; // RS = 1 (data register)
    else   GPIOA->BSRR = GPIO_BSRR_BR0; // RS = 0 (instruction register)

    // send 4 bits to PA2-PA5 outputs
    GPIOA->ODR = (GPIOA->ODR & ~(0xF << 2)) | ((nibble & 0x0F) << 2);
    LCD_Pulse();
}

// send command to LCD
void LCD_Cmd(uint8_t cmd) {
    LCD_SendNibble(cmd >> 4, 0); // high 4th bits
    LCD_SendNibble(cmd & 0x0F, 0); // low 4th bits
    if (cmd == 0x01 || cmd == 0x02) delay_us(2000); // Delay for operations
}

// send 8-bit data to LCD Screen
void LCD_Data(uint8_t data) {
    LCD_SendNibble(data >> 4, 1); //high 4th bits

```

```

    LCD_SendNibble(data & 0x0F, 1); // low 4th bits
}

// set cursor on screen
void LCD_SetCursor(uint8_t col, uint8_t row) {
    cur_row = row % 4; // max 4 rows
    cur_col = col % 20; // max 20 columns
    // compute offset and send command for select current position
    LCD_Cmd(0x80 | (cur_col + row_addresses[cur_row]));
}

//init LCD HD44780
void LCD_Init(void) {
    // enable GPIOA clock
    RCC->APB2ENR |= RCC_APB2ENR_IOPAEN;

    // configure PA0-PA5 to Output Push-Pull 10 MHz
    // clear
    GPIOA->CRL &= ~(0x0FFFFFFF);
    // MODE=01 CNF=00
    GPIOA->CRL |= (0x00111111);

    // wait for LCD
    delay_us(20000);

    // set LCD to 8-bits mode
    LCD_SendNibble(0x03, 0);
    delay_us(5000);
    LCD_SendNibble(0x03, 0);
    delay_us(200);
    LCD_SendNibble(0x03, 0);

    // and then set 4-bit mode
    LCD_SendNibble(0x02, 0);

    // 4. select screen mode
    LCD_Cmd(0x28); // 4 lines, 5x8 chars
    LCD_Cmd(0x0C); // display enable, cursor disable
    LCD_Cmd(0x06); // autoincrement
    LCD_Cmd(0x01); // clear display
}

// print string to LCD
void LCD_Print(const char* str) {
    while(*str) {
        // increment line
        if (cur_col >= 20) {

```

```

        cur_col = 0;
        cur_row = (cur_row + 1) % 4;
        LCD_SetCursor(cur_col, cur_row);
    }
    LCD_Data(*str++);
    cur_col++;
}
}

// backspace
void LCD_Backspace(uint8_t n) {
    for (uint8_t i = 0; i < n; i++) {
        if (cur_col == 0) {
            cur_row = (cur_row == 0) ? 3 : cur_row - 1;
            cur_col = 19;
        } else {
            cur_col--;
        }
        LCD_SetCursor(cur_col, cur_row);
        LCD_Data(' ');
        LCD_SetCursor(cur_col, cur_row);
    }
}

// clean LCD and reset stored position
void LCD_Clear() {
    LCD_Cmd(0x01);
    cur_row = 0;
    cur_col = 0;
    LCD_SetCursor(cur_col, cur_row);
}

// init calculator keypad
void Keypad_Init(void) {

    // enable GPIOB clock
    RCC->APB2ENR |= RCC_APB2ENR_IOPBEN;

    // 3: MODE = 11 (Output 50MHz), CNF = 00 (Push-Pull)
    // 8: MODE = 00 (Input mode), CNF = 10 (Input with pull-up / pull-down)
    GPIOB->CRL &= ~0xFFFFFFFF; //clear 0-7
    GPIOB->CRL |= 0x88833333;

    GPIOB->CRH &= ~0x0000000F; //clear 8
    GPIOB->CRH |= 0x00000008;
}

```

```

        GPIOB->ODR |= (0xF << 5); // set 5-8 pins to 1, on pressed button it changes to
0
    }

    // get pressed key
    uint8_t Scan_Keypad(void) {
        for (uint8_t r = 0; r < 5; r++) {
            // all lines except current set to 1
            GPIOB->ODR = (GPIOB->ODR | 0x1F) & ~(1 << r);
            // get values of cols
            uint16_t cols = (GPIOB->IDR >> 5) & 0x0F;

            // if any cols down (keys pressed)
            if (cols != 0x0F) {
                for (uint8_t c = 0; c < 4; c++) {
                    // if column=0, key on r,c pressed
                    if (!(cols & (1 << c))) {
                        return (r * 4) + c;
                    }
                }
            }
        }

        // no keys pressed
        return 0xFF;
    }

    // get string for key by index
    char* Get_Key_Char(uint8_t index) {
        return keypad_chars[index];
    }

    // clean command buffer
    void Clean_Cmd_Buffer() {
        for (int i = 0; i < 80; i++) {
            cmd_buffer[i] = 0;
        }
        cmd_buffer_pos = 0;
    }

    // add data to command buffer
    void Copy_To_Cmd_Buffer(char *str) {
        while (*str) {
            if (cmd_buffer_pos < 80) {
                cmd_buffer[cmd_buffer_pos++] = (*str++);
            }
        }
    }
}

```

```

#define UART_BAUD_9600 0x341

void UART3_Init(void) {
    // enable clock GPIOB, AFIO (alternate functions), USART3
    SET_BIT(RCC->APB2ENR, RCC_APB2ENR_IOPBEN | RCC_APB2ENR_AFIOEN);
    SET_BIT(RCC->APB1ENR, RCC_APB1ENR_USART3EN);

    // PB10 (TX) - Alt. Function Push-Pull
    MODIFY_REG(GPIOB->CRH, (GPIO_CRH_MODE10 | GPIO_CRH_CNF10),
                (GPIO_CRH_MODE10_1 | GPIO_CRH_MODE10_0 |
GPIO_CRH_CNF10_1));

    // PB11 (RX) - Input Floating
    MODIFY_REG(GPIOB->CRH, (GPIO_CRH_MODE11 | GPIO_CRH_CNF11),
                (GPIO_CRH_CNF11_0));

    // 9600 baud = 52.1 - baud rate register
    // br: 12+4 bits
    WRITE_REG(USART3->BRR, UART_BAUD_9600);

    // configuration register: usart enable, transmitter enable, receiver
enable
    SET_BIT(USART3->CR1, USART_CR1_UE | USART_CR1_TE | USART_CR1_RE);
}

// send char to UART
void UART3_SendChar(char c) {
    // check in status register if transmit data register empty
    while (READ_BIT(USART3->SR, USART_SR_TXE) == 0);
    // write to data register
    WRITE_REG(USART3->DR, c);
}

// receive char to uart
char UART3_ReceiveChar(void) {
    // check in status register if read data register NOT empty
    while (READ_BIT(USART3->SR, USART_SR_RXNE) == 0);
    // read from data register
    return (char)(USART3->DR & 0xFF);
}

// send string to UART
void UART3_SendString(const char* str) {
    while (*str) UART3_SendChar(*str++);
}

```

```

// check if uart3 has income data
char UART3_HasIncomeData() {
    if (READ_BIT(USART3->SR, USART_SR_RXNE) == 0) {
        return 0;
    }
    else {
        return 1;
    }
}

// send line to uart
void UART3_SendLine(const char* str) {
    UART3_SendString(str);
    UART3_SendString("\r\n");
}

// get line from uart3
void UART3_ReceiveLine(char* buffer, uint16_t buffer_size) {
    char prev_sym = 0;
    while (1) {
        char sym = UART3_ReceiveChar();

        if (sym != '\r' && sym != '\n') {
            *buffer++=sym;
            if (--buffer_size < 1) break;
        }

        if (sym == '\n' && prev_sym == '\r') {
            break;
        }

        prev_sym = sym;
    }

    // clear buffer
    while (buffer_size--) *buffer++='\0';
}

// calc expression and get result in buffer
uint8_t CalculateExpr(
    char* expr,
    uint16_t expr_size,
    char* res_buffer,
    uint16_t buf_size) {

```

```

    UART3_SendLine(expr);
    UART3_ReceiveLine(res_buffer, buf_size);

    return 0;
}

int main(void) {
    // init block
    LCD_Init();
    Keypad_Init();
    UART3_Init();

    LCD_Print("CALCULATOR :D");

    // buffer for key strings
    char* ops = "00";

    while(1) {
        uint8_t key = Scan_Keypad();

        if (key != 0xFF) {
            // wait while key released
            while (key ==
Scan_Keypad());

            switch (key) {

                // clear screen and command buffer
                case KEYPAD_CLEAR:

                    LCD_Clear();
                    Clean_Cmd_Buffer();

                    break;

                // eq pressed; compute expression and print
                case KEYPAD_EQ:

                    if (cmd_buffer_pos > 0) {
                        app_state = STATE_SEND_COMMAND;
                        LCD_Clear();

                        uint8_t res = CalculateExpr(cmd_buffer,
cmd_buffer_pos, res_buffer, RES_BUFFER_LEN);
                        Clean_Cmd_Buffer();

```



```

        if (res != 0) {
            app_state =
STATE_ERROR;

            LCD_Print("ERROR");
        }
        else

        {
            app_state = STATE_SHOW_RESULT;
            LCD_Print(res_buffer);
        }
    }

    break;

    // regular key - user input
    default:

        // switch state to input
        if (app_state != STATE_INPUT) {
            LCD_Clear();
            Clean_Cmd_Buffer();
            app_state = STATE_INPUT;
        }

        // get key string and write to lcd and command
buffer

        ops = Get_Key_Char(key);
        LCD_Print(ops);
        Copy_To_Cmd_Buffer(ops);
    }

    // wait before read next key
    delay_us(5000);
}

// check if has income data on uart
if (UART3_HasIncomeData()) {
    // print data from uart on screen
    app_state = STATE_PRINT_HISTORY;
    UART3_ReceiveLine(res_buffer, RES_BUFFER_LEN);
    LCD_Clear();
    Clean_Cmd_Buffer();
    LCD_Print(res_buffer);
}
}
}

```

## A.2 Код сервера (CalculatorServer.pde)

```
import processing.serial.*;

// ===== CONFIGURATION =====
String PORT_NAME = "COM2";
int BAUD_RATE = 9600;

// ===== GLOBAL STATE =====
Serial port;
boolean portConnected = false;
String lastReceived = "";
String lastSent = "";
String statusMessage = "Starting...";

ArrayList<JournalEntry> journal = new ArrayList<JournalEntry>();
int scrollOffset = 0;
int maxVisibleEntries = 10;

// ===== JOURNAL ENTRY =====
class JournalEntry {
    String expression;
    String result;
    String timestamp;

    JournalEntry(String expr, String res) {
        this.expression = expr;
        this.result = res;
        this.timestamp = nf(hour(), 2) + ":" + nf(minute(), 2) + ":" + nf(second(), 2);
    }

    String display() {
        return expression + " = " + result;
    }
}

// ===== SETUP =====
void setup() {
    size(500, 600);
    textFont(createFont("Monospaced", 14));

    // List available ports
    println("Available serial ports:");
    printArray(Serial.list());
}
```

```

    // Try to connect
    connectSerial();
}

void connectSerial() {
    try {
        port = new Serial(this, PORT_NAME, BAUD_RATE);
        port.bufferUntil('\n');
        portConnected = true;
        statusMessage = "Connected to " + PORT_NAME;
        println(statusMessage);
    } catch (Exception e) {
        portConnected = false;
        statusMessage = "Failed to connect: " + PORT_NAME;
        println(statusMessage);
        println("Available ports: ");
        printArray(Serial.list());
    }
}

// ===== SERIAL EVENT =====
void serialEvent(Serial p) {
    String raw = p.readStringUntil('\n');
    if (raw == null) return;

    // Remove \r\n
    String input = raw.trim();
    if (input.length() == 0) return;

    lastReceived = input;
    println("RX: " + input);

    // Parse and calculate
    String result;
    try {
        double value = parseExpression(input);
        // Format result: remove trailing zeros
        if (value == (long) value) {
            result = String.valueOf((long) value);
        } else {
            result = String.format("%.6f", value).replaceAll("0+$",
"").replaceAll("\\.0$", "");
        }
    } catch (Exception e) {
        result = "Error: " + e.getMessage();
    }
}

```

```

    // Add to journal
    journal.add(new JournalEntry(input, result));

    // Send result back
    sendResult(result);
}

void sendResult(String result) {
    if (!portConnected) return;

    port.write(result + "\r\n");
    lastSent = result;
    println("TX: " + result);
}

void sendToClient(String message) {
    if (!portConnected) return;

    port.write(message + "\r\n");
    lastSent = message;
    println("TX (push): " + message);
}

// ===== EXPRESSION PARSER =====
// Recursive descent parser
// Grammar:
//   expr  -> term (('+' | '-') term)*
//   term  -> factor (('*' | '/') factor)*
//   factor -> 'ln(' expr ')' | '(' expr ')' | number

int pos;
String expr;

double parseExpression(String input) throws Exception {
    expr = input.replaceAll("\\s+", ""); // remove whitespace
    pos = 0;
    double result = parseExpr();

    if (pos < expr.length()) {
        throw new Exception("syntax");
    }
    return result;
}

double parseExpr() throws Exception {
    double left = parseTerm();

```

```

while (pos < expr.length()) {
    char op = expr.charAt(pos);
    if (op == '+') {
        pos++;
        left = left + parseTerm();
    } else if (op == '-') {
        pos++;
        left = left - parseTerm();
    } else {
        break;
    }
}
return left;
}

double parseTerm() throws Exception {
    double left = parseFactor();

    while (pos < expr.length()) {
        char op = expr.charAt(pos);
        if (op == '*') {
            pos++;
            left = left * parseFactor();
        } else if (op == '/') {
            pos++;
            double right = parseFactor();
            if (right == 0) throw new Exception("div/0");
            left = left / right;
        } else {
            break;
        }
    }
    return left;
}

double parseFactor() throws Exception {
    // Check for ln(
    if (pos + 2 < expr.length() &&
        expr.charAt(pos) == 'l' &&
        expr.charAt(pos + 1) == 'n' &&
        expr.charAt(pos + 2) == '(') {
        pos += 3; // skip "ln("
        double arg = parseExpr();
        if (pos >= expr.length() || expr.charAt(pos) != ')') {
            throw new Exception("syntax");
        }
    }
}

```

```

        pos++; // skip ')'
        if (arg <= 0) throw new Exception("ln domain");
        return Math.log(arg);
    }

    // Check for '('
    if (pos < expr.length() && expr.charAt(pos) == '(') {
        pos++; // skip '('
        double result = parseExpr();
        if (pos >= expr.length() || expr.charAt(pos) != ')') {
            throw new Exception("syntax");
        }
        pos++; // skip ')'
        return result;
    }

    // Check for unary minus
    boolean negative = false;
    if (pos < expr.length() && expr.charAt(pos) == '-') {
        negative = true;
        pos++;
    }

    // Parse number
    int start = pos;
    while (pos < expr.length() && (Character.isDigit(expr.charAt(pos)) ||
expr.charAt(pos) == '.')) {
        pos++;
    }

    if (start == pos) {
        throw new Exception("syntax");
    }

    double value = Double.parseDouble(expr.substring(start, pos));
    return negative ? -value : value;
}

// ===== DRAW =====
void draw() {
    background(30);

    // Header
    fill(portConnected ? color(100, 255, 100) : color(255, 100, 100));
    textAlign(LEFT, TOP);
    text(statusMessage + " [" + BAUD_RATE + " baud]", 20, 20);

```

```
// Divider
stroke(100);
line(20, 50, width - 20, 50);

// Last RX/TX
fill(200);
text("Last RX: " + lastReceived, 20, 60);
text("Last TX: " + lastSent, 20, 80);

// Divider
line(20, 110, width - 20, 110);

// Journal header
fill(255);
textSize(16);
text("Journal (" + journal.size() + " entries)", 20, 120);
textSize(14);

// Journal list
int y = 150;
int entryHeight = 35;

for (int i = scrollOffset; i < min(journal.size(), scrollOffset +
maxVisibleEntries); i++) {
    JournalEntry entry = journal.get(i);

    // Entry background
    fill(50);
    noStroke();
    rect(20, y, width - 80, entryHeight - 5, 5);

    // Entry text
    fill(220);
    textAlign(LEFT, CENTER);
    text((i + 1) + ". " + entry.display(), 30, y + entryHeight/2 - 2);

    // Timestamp
    fill(120);
    textSize(10);
    text(entry.timestamp, 30, y + entryHeight - 8);
    textSize(14);

    // Send button
    fill(70, 130, 180);
    rect(width - 55, y, 35, entryHeight - 5, 5);
    fill(255);
    textAlign(CENTER, CENTER);
```

```

        text(">", width - 37, y + entryHeight/2 - 2);

        y += entryHeight;
    }

    // Scroll indicators
    if (journal.size() > maxVisibleEntries) {
        fill(150);
        textAlign(CENTER, BOTTOM);
        if (scrollOffset > 0) {
            text("^ Scroll up ^", width/2, 145);
        }
        textAlign(CENTER, TOP);
        if (scrollOffset + maxVisibleEntries < journal.size()) {
            text("v Scroll down v", width/2, y + 10);
        }
    }

    // Instructions
    fill(100);
    textAlign(LEFT, BOTTOM);
    text("Click [>] to send result to client | Scroll: mouse wheel | R: reconnect",
20, height - 10);
}

// ===== MOUSE =====
void mousePressed() {
    int y = 150;
    int entryHeight = 35;

    for (int i = scrollOffset; i < min(journal.size(), scrollOffset +
maxVisibleEntries); i++) {
        // Check if click is on send button
        if (mouseX >= width - 55 && mouseX <= width - 20 &&
            mouseY >= y && mouseY <= y + entryHeight - 5) {

            JournalEntry entry = journal.get(i);
            sendToClient(entry.result);
            return;
        }
        y += entryHeight;
    }
}

void mouseWheel(MouseEvent event) {
    int e = event.getCount();

```



```
        scrollOffset = constrain(scrollOffset + e, 0, max(0, journal.size() -
maxVisibleEntries));
    }

    // ===== KEYBOARD =====
    void keyPressed() {
        if (key == 'r' || key == 'R') {
            if (port != null) port.stop();
            connectSerial();
        }
    }
}
```